Project Report

AI-POWERED STOCK SCREENER

Submitted by:
Nam: Dimple Yadav
Reg. no.: 12308071
Section: 40M58
Roll no.: 66

Submitted to:
Ms. Arshiya Pathania

School of Computer Science and Engineering
Lovely Professional University

1. Introduction

1.1. Background and Motivation

The financial markets generate an immense volume of data every second stock prices, earnings reports, sector movements, macroeconomic indicators, and analyst ratings making it increasingly difficult for individual investors and analysts to filter, evaluate, and act on relevant information in a timely manner. Traditional stock screening tools, while functional, demand that users possess a deep familiarity with financial metrics and the technical interfaces of screening platforms. A user wishing to find "healthcare companies with strong revenue growth and a low price-to-earnings ratio" must translate their intent into precise numerical filters across multiple fields, a process that is both time-consuming and inaccessible to a large segment of the investing public.

Simultaneously, the field of software engineering is undergoing a fundamental transformation. The rapid adoption of DevOps practices, a cultural and technical movement that unifies software development and IT operations has redefined how modern applications are built, deployed, and maintained. Infrastructure is no longer configured by hand on physical servers; it is described in code, version-controlled, and provisioned automatically. Applications are no longer deployed as monolithic binaries; they are decomposed into independently deployable microservices, packaged as containers, and orchestrated at scale across cloud infrastructure.

This project sits at the intersection of these two transformations. The **AI-Powered Stock Screener** is designed as a demonstration of how modern AI capabilities specifically, Google's Gemini Large Language Model, can be combined with a robust, production-grade DevOps pipeline to deliver a sophisticated, scalable financial tool. Users can interact with the system through plain English queries such as *"Show me technology companies with a PE ratio below 30"*, and the system intelligently translates this natural language intent into a structured database query, returning relevant financial data in seconds.

Critically, the application is not deployed as a simple local script. It is designed and deployed as a cloud-native system on Amazon Web Services (AWS), managed by a complete DevOps toolchain encompassing Infrastructure as Code (Terraform), containerization (Docker), container orchestration (Kubernetes), automated CI/CD pipelines (Jenkins), and comprehensive observability (Prometheus and Grafana). This holistic approach makes the project a real-world representation of how modern engineering teams build and operate software at scale.

1.2. Problem Statement

Despite the wealth of financial data available to the public, several key challenges prevent its effective use:

1. **Complexity of Traditional Screeners**: Existing tools like Bloomberg Terminal or Yahoo Finance screeners require users to understand and manually configure dozens of financial parameters (e.g., P/E ratios, EPS, EBITDA margins). This creates a high barrier to entry for non-expert users.
2. **Static, Non-Conversational Interfaces**: Traditional screeners are form-based and rigid. They cannot understand user *intent* — a query like "find undervalued tech stocks" cannot be entered directly; it must be manually decomposed into individual filter conditions.
3. **Operational Complexity in Deployment**: Building a scalable, resilient financial application requires significant infrastructure knowledge. Without automation, provisioning servers, deploying updates, and monitoring system health are all manual, error-prone processes that do not scale.
4. **Gap Between Development and Operations**: In many projects, the application code and the infrastructure that runs it are treated as separate concerns. This disconnect leads to deployment failures, configuration drift, and difficulty in diagnosing production issues.

This project directly addresses all four challenges: the AI layer solves the accessibility and intent-understanding problem, while the DevOps pipeline solves the operational complexity and development-operations gap.

1.3. Project objective

The primary objectives of this project are as follows:

1. **To design and implement an AI-powered Natural Language Interface** for stock screening, leveraging the Google Gemini API to translate free-text queries into structured database queries via a custom Domain-Specific Language (DSL).
2. **To architect a microservices-based backend** using FastAPI, where distinct services handle authentication, query processing, portfolio management, alert management, and LLM communication independently.
3. **To automate cloud infrastructure provisioning** on AWS using Terraform, ensuring that the entire environment — from the Virtual Private Cloud (VPC) to compute instances — can be created and destroyed reproducibly through code.
4. **To containerize all application services** using Docker, ensuring environment consistency across development, staging, and production.
5. **To orchestrate the containerized workloads** at scale using a self-managed Kubernetes cluster provisioned on AWS EC2 instances, demonstrating pod scheduling, service discovery, and secrets management.
6. **To implement a fully automated CI/CD pipeline** using Jenkins, which automatically builds Docker images, pushes them to Docker Hub, and deploys updated manifests to the Kubernetes cluster upon every code commit.

7. **To establish a comprehensive observability stack** using Prometheus for metrics collection and Grafana for visualization, providing real-time insight into both application-level and infrastructure-level health.

1.4. Scope of The Project

This project encompasses the full-stack development and end-to-end DevOps implementation of the AI Stock Screener application. The scope includes:

- **Application Layer**: A Streamlit-based frontend, a FastAPI backend, a dedicated LLM microservice, a PostgreSQL database, a Redis caching layer, and a data synchronization worker.
- **Infrastructure Layer**: AWS VPC, public subnets, security groups, internet gateways, and four EC2 instances (one Kubernetes master, two Kubernetes workers, one Jenkins server).
- **DevOps Toolchain**: Terraform for IaC, Docker for containerization, Kubernetes (via Kubeadm) for orchestration, Jenkins for CI/CD automation, and the Prometheus and Grafana stack for monitoring.

The scope explicitly **excludes** real-time trading capabilities, financial advice generation, and integration with live brokerage APIs. The stock data used in the system is sourced from the Yahoo Finance API (via the `yfinance` Python library) and is intended for informational and demonstration purposes only.

2. DevOps Methodology and ToolChain Selection

2.1. The DevOps Culture and Lifecycle

DevOps is a set of cultural practices, methodologies, and tools that aims to bridge the traditional gap between software development (Dev) and IT operations (Ops) teams. Rather than treating application development and infrastructure management as separate and sequential concerns, DevOps promotes a continuous, collaborative lifecycle where code is continuously integrated, tested, delivered, and monitored in an automated fashion.

The core philosophy of DevOps is built around the **CALMS** framework:

- **Culture**: Shared responsibility between developers and operations engineers.
- **Automation**: Replacing manual, error-prone processes with scripted, repeatable pipelines.

- **Lean**: Eliminating waste through faster feedback loops and smaller, more frequent deployments.
- **Measurement**: Using metrics and monitoring to make data-driven decisions.
- **Sharing**: Transparency of tools, processes, and outcomes across teams.

In this project, the DevOps lifecycle is implemented across three continuous loops:

1. **Continuous Integration (CI)**: Every code push to the GitHub repository automatically triggers a Jenkins pipeline that builds a new Docker image, verifying that the code compiles and packages correctly.
2. **Continuous Deployment (CD)**: Upon a successful build, Jenkins automatically pushes the new image to Docker Hub and applies the updated Kubernetes manifests to the live cluster on AWS.
3. **Continuous Monitoring**: Prometheus continuously scrapes metrics from all running services, and Grafana displays this data in real-time dashboards, ensuring any degradation in performance or availability is immediately visible.

This end-to-end automated lifecycle ensures that a developer can push a code change and have it live in the cloud-based Kubernetes environment within minutes, without any manual intervention.

2.2. Infrastructure as Code (IaC) — Why Terraform?

Before the adoption of Infrastructure as Code, provisioning cloud infrastructure was a manual process carried out through web consoles or CLI commands. This approach is time-consuming, difficult to replicate, and highly susceptible to configuration drift — where the actual state of a server gradually diverges from its intended state due to manual changes.

Terraform, developed by HashiCorp, is the industry-leading open-source IaC tool. It allows infrastructure to be defined using the **HashiCorp Configuration Language (HCL)**, a declarative syntax that describes the desired state of the infrastructure. Terraform then computes the difference between the current state and the desired state and applies only the necessary changes.

For this project, Terraform was chosen over alternatives (such as AWS CloudFormation) for the following reasons:

- **Cloud-Agnostic**: Terraform supports over 1,000 cloud and service providers through its plugin-based provider system, making the infrastructure knowledge transferable beyond AWS.
- **State Management**: Terraform maintains a `.tfstate` file that acts as a precise record of all provisioned resources, enabling accurate planning and preventing duplicate resource creation.
- **Declarative & Idempotent**: Running `terraform apply` multiple times always converges to the same desired state, making infrastructure management predictable and safe.
- **Modularity**: Complex infrastructure can be broken into reusable modules. In this project, the VPC, subnets, security groups, and EC2 instances are each defined in separate, focused `.tf` files for clarity and maintainability.

2.3. Containerization — Why Docker?

A fundamental challenge in software deployment is the "works on my machine" problem — an application that runs correctly in a developer's local environment may fail in production due to differences in operating system versions, installed libraries, or environment variables.

Docker solves this problem by packaging an application together with all its dependencies, runtime, and configuration into a portable unit called a **container**. Unlike a virtual machine, which virtualizes an entire operating system, a Docker container shares the host OS kernel and is therefore significantly lighter, faster to start, and more resource-efficient.

Docker was selected for this project because:

- **Reproducible Builds**: The `Dockerfile` acts as a precise recipe for building the application image. Any developer or CI server can produce an identical image from the same `Dockerfile`.
- **Isolation**: Each microservice (frontend, backend, LLM service, data sync worker) runs in its own container with no interference from other services.
- **Docker Hub Integration**: Built images are published to Docker Hub, creating a versioned, centrally accessible registry that Kubernetes can pull from on any node in the cluster.
- **Microservice Suitability**: The multi-service architecture of the AI Stock Screener maps perfectly onto Docker's container model, where each service has its own Dockerfile and image lifecycle.

2.4. Orchestration — Why Kubernetes?

Running containers on a single machine using Docker alone is insufficient for a production-grade application. If a container crashes, it must be manually restarted. If traffic increases, there is no mechanism to automatically scale up. If the host machine fails, all services become unavailable.

Kubernetes (K8s) is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications across a cluster of machines. It was chosen for this project over simpler alternatives (such as Docker Compose) for the following reasons:

- **Self-Healing**: Kubernetes continuously monitors the health of pods (its smallest deployable unit). If a container crashes, Kubernetes automatically restarts it, maintaining the desired replica count without human intervention.
- **Declarative Configuration**: Kubernetes workloads are defined in YAML manifests that describe the desired state. This integrates seamlessly with the project's Git-based workflow.
- **Service Discovery**: Kubernetes provides built-in DNS-based service discovery. The backend service can reach the database simply using the hostname `db`, without needing to know the database pod's IP address.
- **Horizontal Scaling**: The backend is configured with 2 replicas, and Kubernetes automatically load-balances requests across both pods, improving throughput and resilience.
- **Secrets Management**: Kubernetes Secrets store sensitive data (such as the Gemini API key) separately from application code, injecting them into containers at runtime as environment variables.

The Kubernetes cluster in this project is bootstrapped using **kubeadm** on AWS EC2 instances — a self-managed approach that demonstrates a deep understanding of the Kubernetes control plane and worker node architecture, as opposed to using a managed service like EKS.

2.5. CI/CD and Monitoring — Why Jenkins, Prometheus & Grafana?

Jenkins

Jenkins is the world's most widely adopted open-source automation server. It was selected as the CI/CD engine for this project because of its extensive plugin ecosystem, its seamless integration with Git webhooks, and its native support for **Declarative Pipelines** defined as `Jenkinsfile` — a file committed directly to the repository alongside application code.

The Jenkins server runs on a dedicated EC2 instance within the same AWS VPC as the Kubernetes cluster, ensuring low-latency communication during the deployment stage. The pipeline is structured in four clearly defined stages: **Checkout** → **Build** → **Push** → **Deploy**, providing a transparent and auditable record of every deployment event.

Prometheus & Grafana

Prometheus is an open-source monitoring and alerting toolkit originally built at SoundCloud and now a graduated project of the Cloud Native Computing Foundation (CNCF). It follows a **pull-based model**, where it periodically scrapes metrics from a configured list of targets that expose a `/metrics` HTTP endpoint.

In this project, Prometheus is deployed via the **kube-prometheus-stack** Helm chart, which bundles Prometheus, the Alertmanager, and a set of pre-built Kubernetes monitoring rules. Application-level metrics are exposed automatically by the `prometheus-fastapi-instrumentator` library integrated into both the backend and LLM service.

Grafana is the industry-standard open-source visualization platform, used here to transform the raw time-series data collected by Prometheus into interactive, real-time dashboards. Through the kube-prometheus-stack installation, a rich set of pre-built Kubernetes dashboards (covering API server health, kubelet metrics, pod resource consumption, and network performance) are available out of the box alongside custom application-level dashboards.

The combination of Prometheus and Grafana provides **full-stack observability** — from OS-level CPU and memory on each EC2 node down to the HTTP request rate and response latency of individual FastAPI endpoints.

3. System Architecture and Design

3.1. High-Level System Architecture

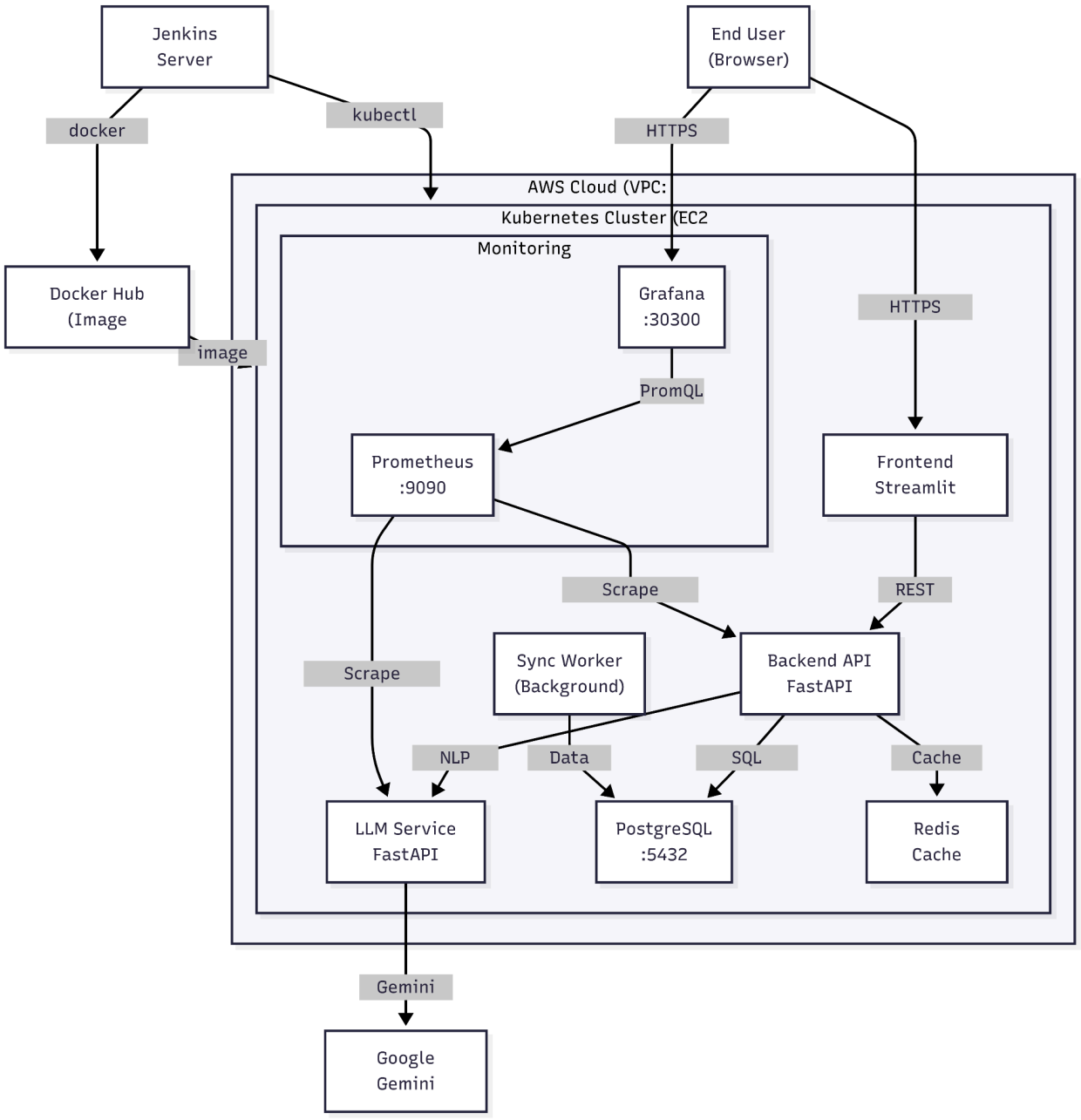
The AI Stock Screener is built on a **microservices architecture**, where each functional concern of the application is encapsulated in a dedicated, independently deployable service. These services communicate over a well-defined network within a Kubernetes cluster hosted on AWS EC2 instances.

The system comprises seven distinct services:

Service	Technology	Role
Frontend	Streamlit (Python)	User interface — query input, portfolio dashboard, alerts
Backend API	FastAPI (Python)	Core business logic, authentication, request routing
LLM Service	FastAPI + Gemini API	Translates natural language to structured DSL JSON
PostgreSQL	PostgreSQL 15	Persistent relational data store for all application data
Redis	Redis 7	In-memory caching layer to accelerate repeated queries
Sync Worker	Python (yfinance)	Background job that fetches and updates financial data
Monitoring Stack	Prometheus + Grafana	Metrics collection and visualization

All seven services run as Kubernetes pods on a three-node cluster (one master, two workers) provisioned on AWS EC2 using Terraform. External users access the frontend through a Kubernetes NodePort service exposed on port 30501, while Grafana dashboards are accessible on port 30300.

► **Diagram 1: High-Level System Architecture**



3.2. Microservices Interaction Model

The communication flow between microservices follows a strict request-response chain. When a user enters a natural language query, it travels through three distinct service boundaries before a result is returned:

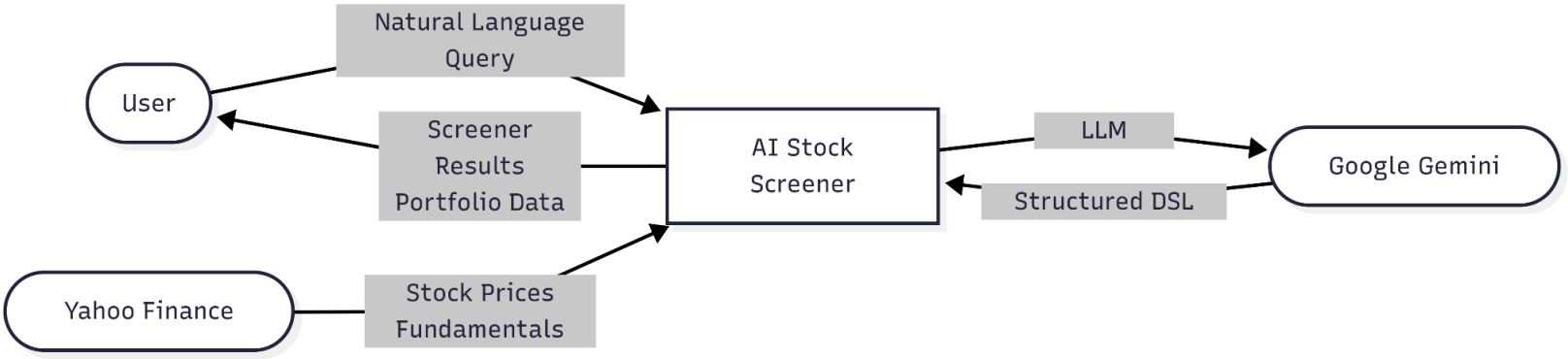
1. The **Frontend** captures the user's plain-English query and sends it as an authenticated HTTP POST request to the Backend API's `/query/` endpoint.
2. The **Backend API** first checks Redis for a cached result. On a cache miss, it forwards the query text to the LLM Service.
3. The **LLM Service** constructs a carefully engineered prompt and submits it to the Google Gemini API. The response is a structured JSON object conforming to a custom Domain Specific Language (DSL) schema.
4. The **Backend API** receives this DSL object, validates it, and passes it to the SQL Compiler, which generates a safe, parameterized SQL query. This query is executed against PostgreSQL.
5. The results are cached in Redis (with a configurable TTL) and returned to the Frontend for display.

All inter-service communication within the cluster is handled via Kubernetes ClusterIP services, which provide stable DNS names (e.g., `http://llm-service:8001`) regardless of pod IP changes. No internal service is exposed directly to the public internet.

3.3. Data Flow Diagrams (DFD)

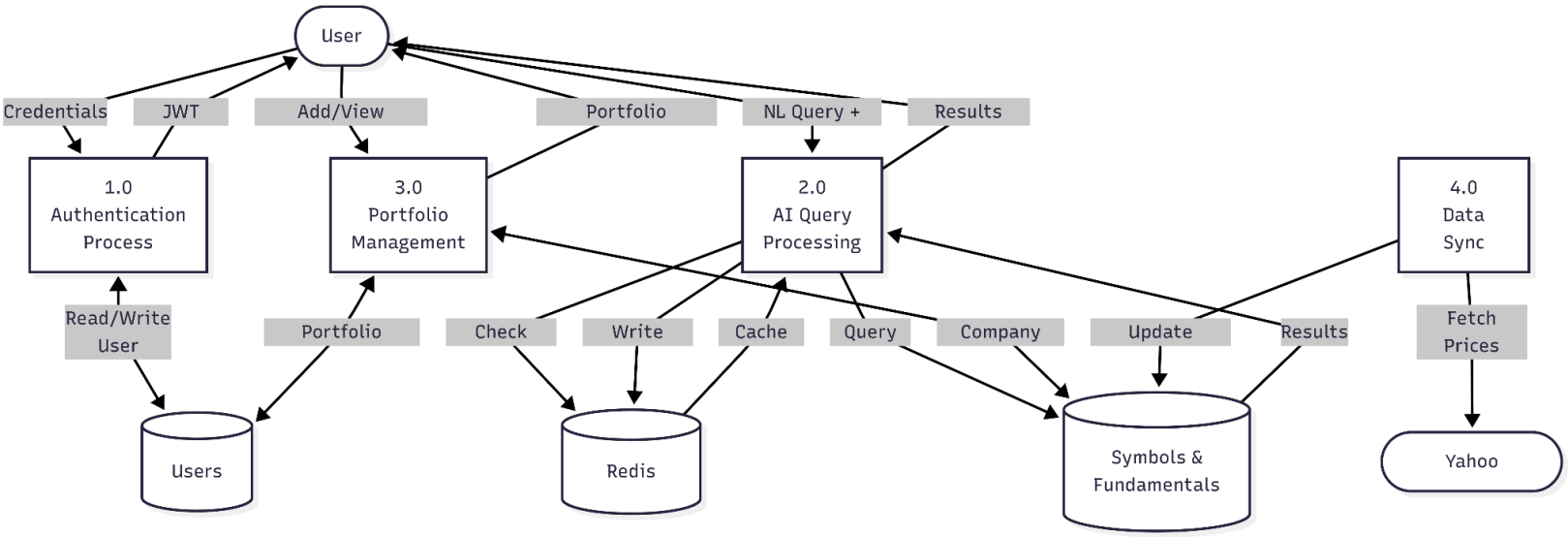
- ### Level 0 DFD — Context Diagram

The Level 0 DFD shows the system as a single process interacting with the key external entities.



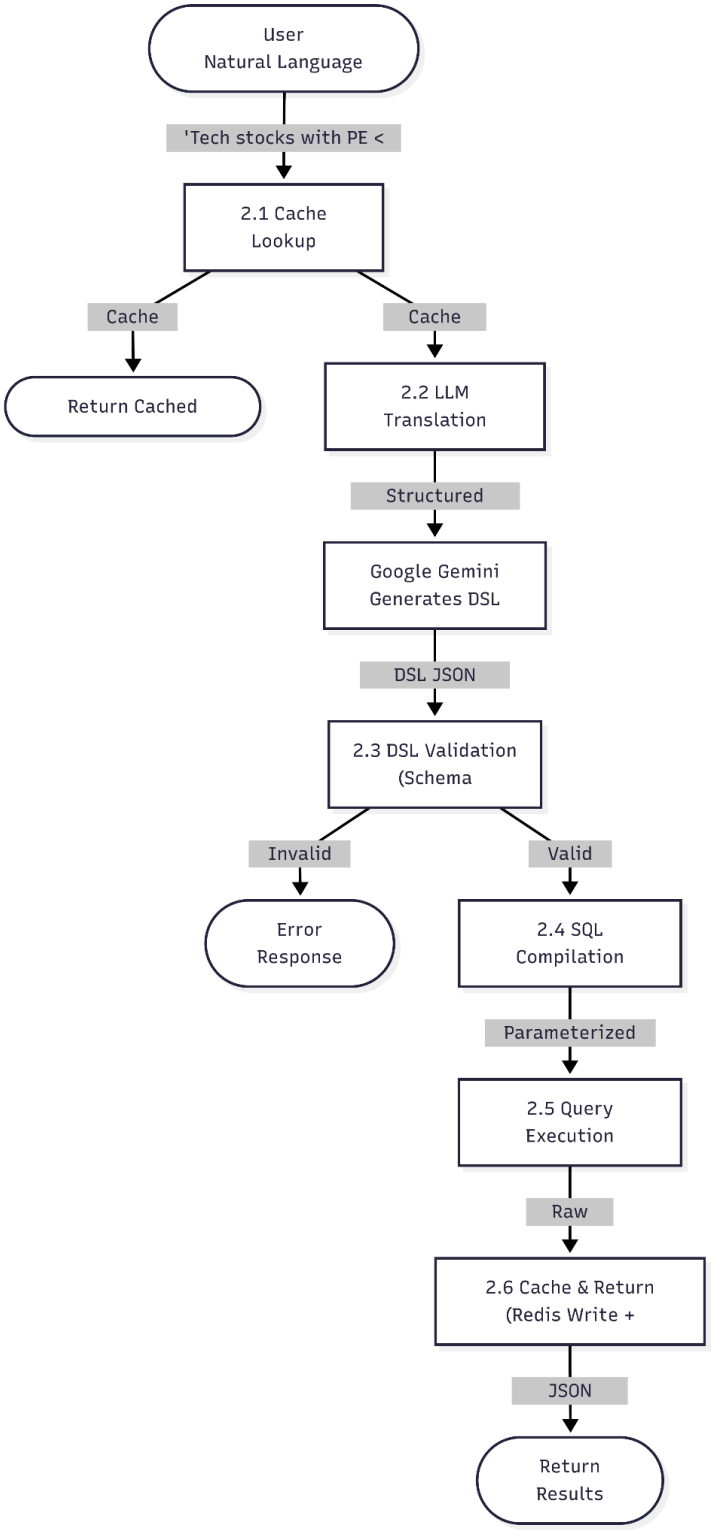
- # Level 1 DFD — Core Processes

  The Level 1 DFD decomposes the system into its four core processes and shows how data flows between them and the data stores.



- ## Level 2 DFD — NLP to SQL Pipeline

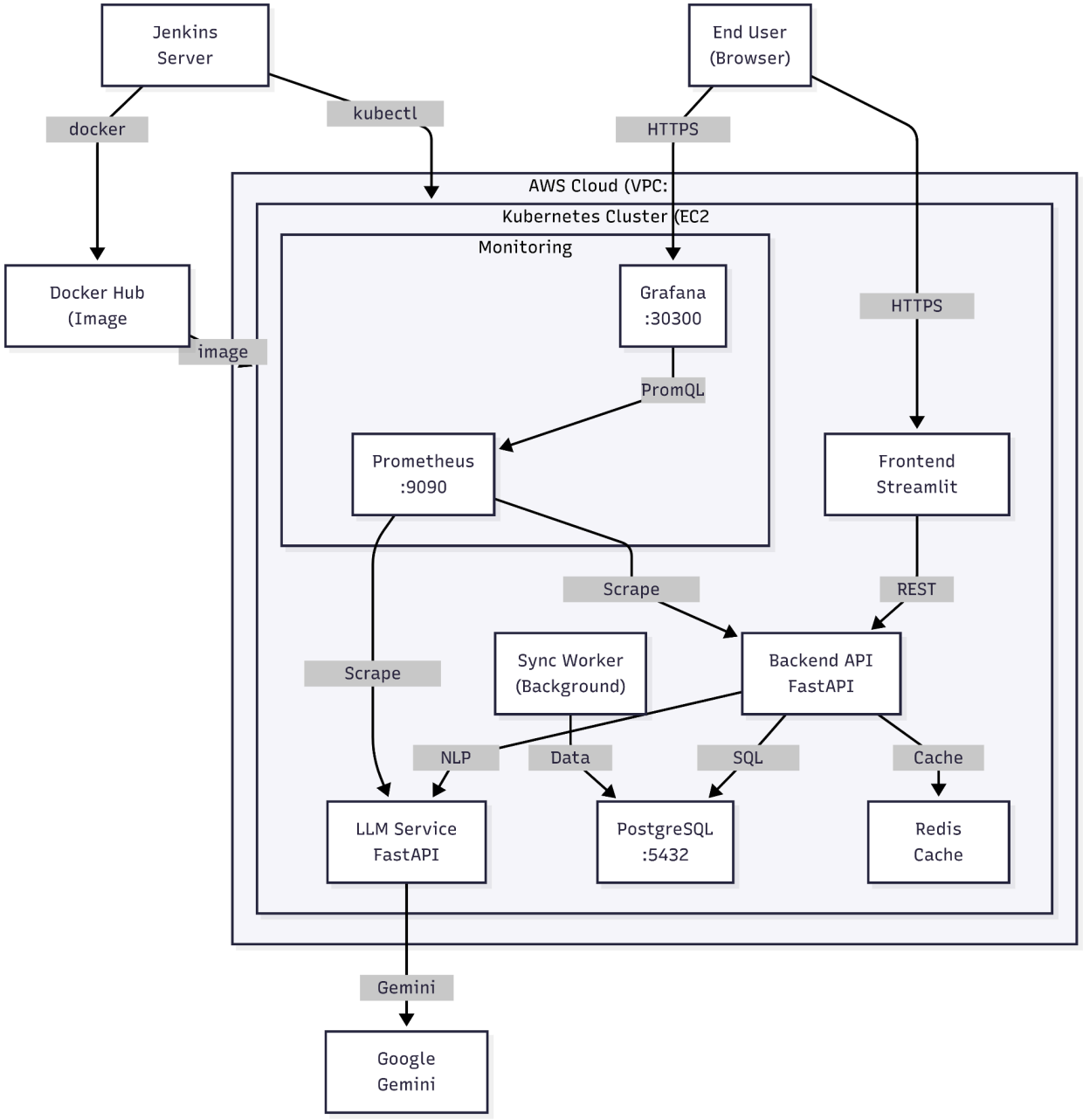
  This diagram zooms into Process 2.0 (AI Query Processing) and shows the detailed internal steps that transform a plain-English user query into database results.



3.4. Kubernetes Deployment Architecture

Beyond the logical application architecture, it is essential to understand the physical deployment layout — specifically, how application services are distributed across the Kubernetes cluster nodes.

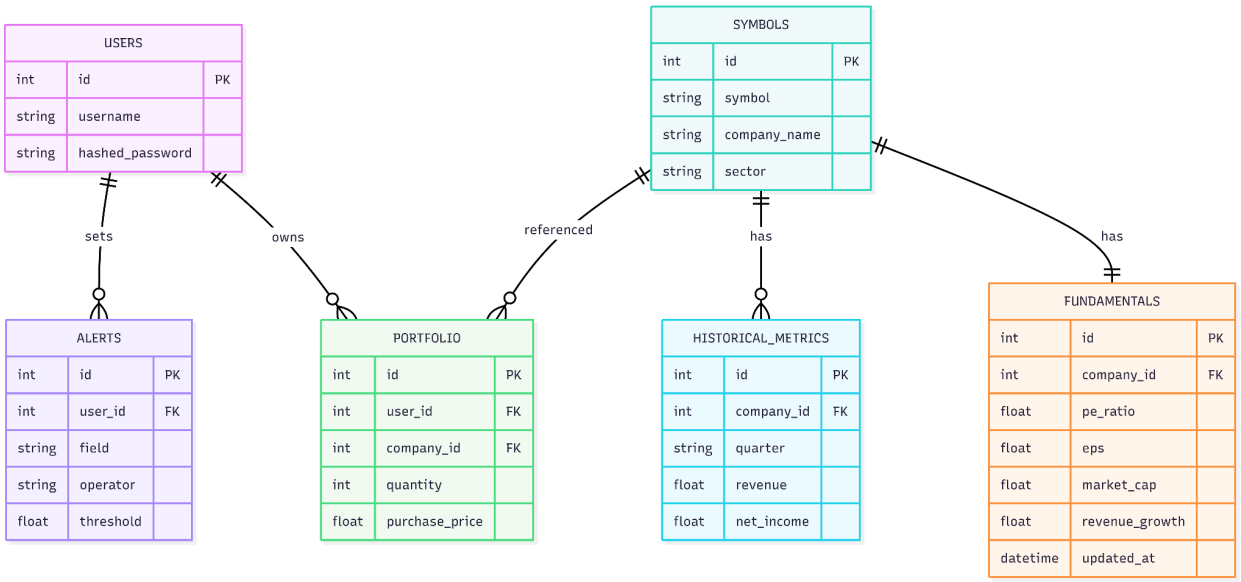
Diagram 5: Kubernetes Deployment Architecture on AWS



3.5. Database Entity-Relationship (ER) Diagram

The PostgreSQL database underpinning the application consists of five tables with clearly defined relationships. The schema is managed by SQLAlchemy ORM and is created automatically on application startup.

Diagram 6: Database ER Diagram



4. Core Application Components

4.1. Frontend Application (Streamlit)

The user-facing interface of the AI Stock Screener is built using **Streamlit**, an open-source Python framework that enables the rapid development of interactive web applications entirely in Python, without requiring any JavaScript or HTML knowledge. While primarily used for data science dashboards, Streamlit is well-suited for this project because the entire application stack is Python-based, ensuring a consistent development environment across all services.

The frontend is containerized and deployed as a Kubernetes pod, accessible to external users via a **NodePort** service on port `30501`. The application URL for backend communication is injected as an environment variable (`API_URL`), making it fully configurable between local development and the Kubernetes deployment.

- ## User Interface & AI Screener Dashboard

Upon successful login, the user is presented with a three-tab dashboard:

Tab 1 — AI Screener: The primary feature of the application. A text input box accepts any free-form, natural language financial query. Upon submission, the query is sent to the backend API, and the results are rendered in three formats:

- A **metrics summary bar** showing the total number of results, the average P/E ratio, and the most common sector.
- A **searchable, sortable data table** containing all matched companies.
- **Two interactive Plotly charts** — a bar chart comparing P/E ratios across matched companies, and a pie chart showing the sector composition of the results.

Tab 2 — Portfolio Management: Displays the user's investment portfolio with real-time market data fetched in a single batch call to the Yahoo Finance API. The dashboard presents:

- Summary cards showing total portfolio market value, unrealized profit/loss, and average performance percentage.
- A **portfolio trend line chart** showing the total portfolio value over the previous 30 days.
- A **holdings details table** with purchase price, current price, and profit/loss percentage per holding.
- A **portfolio heatmap** using Plotly's treemap visualization, where colour-coding from red to green indicates performance at a glance.

Tab 3 — Smart Alerts: Allows users to define threshold-based alerts (e.g., "alert when PE ratio drops below 15") which are stored persistently in the database.

- ## Authentication Flow

The sidebar manages authentication state. User credentials are submitted via a standard login form. Upon a successful response from the backend, a **JWT (JSON Web Token)** is stored in Streamlit's `session_state`, which persists the user's session across page reruns. This token is attached as a `Bearer` token in the `Authorization` header of every subsequent API request, ensuring that all protected endpoints remain inaccessible without a valid, unexpired token.

4.2. Backend REST API (FastAPI)

The backend is built using **FastAPI**, a modern, high-performance Python web framework built on top of **Starlette** and **Pydantic**. FastAPI's key advantages for this project are:

- **Automatic OpenAPI Documentation**: FastAPI generates an interactive `/docs` endpoint (Swagger UI) automatically from function signatures and Pydantic models, providing built-in API documentation at no extra cost.
- **Data Validation**: All incoming request bodies are validated against Pydantic schemas, ensuring type safety and preventing malformed data from reaching business logic.
- **Prometheus Integration**: The `prometheus-fastapi-instrumentator` library is initialized on startup with a single call, automatically exposing a `/metrics` endpoint that Prometheus can scrape.

```
# main.py -- Prometheus instrumentation (one line setup)
from prometheus_fastapi_instrumentator import Instrumentator
Instrumentator().instrument(app).expose(app)
```

The API is organized into five routers, each handling a distinct domain:

Router	Prefix	Endpoints
Auth	/auth	POST /register, POST /login
Query	/query	POST / (AI Screener)
Portfolio	/portfolio	GET /, POST /
Alerts	/alerts	GET /, POST /
Companies	/companies	GET / (full company listing)

Authentication & JWT Security

Authentication is implemented using the **OAuth2 Password Flow** with **JWT tokens**. When a user logs in, the backend verifies their password against the `bcrypt`-hashed value stored in PostgreSQL. On success, a signed JWT is generated using the `python-jose` library and returned to the client.

All protected routes use a `get_current_user` dependency which is injected via FastAPI's `Depends()` mechanism. This dependency extracts the JWT from the request header, verifies the signature using the `SECRET_KEY`, and retrieves the corresponding user record from the database — all before the route handler function is even called.

4.3. AI & NLP Engine — Gemini LLM Integration

The intelligence layer of the application is encapsulated in a dedicated **LLM Microservice**, a separate FastAPI application running in its own Kubernetes pod. This separation of concerns means the AI logic can be scaled, updated, or replaced independently of the rest of the system.

• Natural Language to DSL Translation

The core of the AI engine is a carefully engineered **prompt** sent to Google's Gemini model. The prompt instructs the model to act as a "financial query parser" and defines a strict JSON output schema — the project's custom **Domain Specific Language (DSL)**. The prompt provides the model with the list of allowed database fields, allowed comparison operators, and several few-shot examples to guide accurate output.

```
# parser.py -- DSL JSON schema enforced via prompt engineering
prompt = f"""
You are a financial query parser. Convert the following natural language
query into a structured DSL JSON format.

Allowed fields: {', '.join(ALLOWED_FIELDS)}
Allowed operators: =, <, >, <=, >=

DSL Schema:
{{
  "filters": [{{"field": "string", "operator": "string", "value": number}}],
  "logic": "AND" | "OR",
  "time_filter": {{"type": "quarter", "value": "YYYY-QX"}}
}}

Query: {query}
"""
```

For example, the natural language input "`Technology stocks with PE ratio below 30`" is transformed into the following DSL JSON by the Gemini model:

```
{
  "filters": [
    {"field": "sector", "operator": "=", "value": "Technology"},
    {"field": "pe_ratio", "operator": "<", "value": 30}
  ],
  "logic": "AND"
}
```

- ## DSL Validation & SQL Compilation

Before the DSL JSON is used to construct a database query, it is passed through a **DSL Validator** that checks all field names against a whitelist and verifies that all operators are from an explicitly allowed set. This prevents prompt injection attacks where a malicious user might craft an input designed to trick the LLM into producing a DSL that accesses unauthorized data.

The validated DSL is then passed to the **SQL Compiler** (`sql_compiler.py`), which maps DSL fields to their corresponding database table and column names, constructs the appropriate `JOIN` clauses, and builds a fully parameterized SQL query. Using parameterized queries (via SQLAlchemy's `text()` function with a parameter dictionary) is the industry-standard defence against SQL injection attacks.

```
# sql_compiler.py -- Parameterized query construction
where_clauses.append(f"{column} {filter.operator} :p_{len(params)}")
params.append(filter.value)
# Parameters are passed separately, never interpolated into the string
```

4.4. Data Layer — PostgreSQL & Redis Caching

- ## PostgreSQL (Persistent Storage)

The primary data store is **PostgreSQL**, a production-grade open-source relational database. The schema, defined using SQLAlchemy's declarative ORM (`models.py`), consists of five tables: `users`, `symbols`, `fundamentals`, `historical_metrics`, and `portfolio`. The schema is version-controlled alongside the application code and created automatically on startup via `Base.metadata.create_all()`.

Financial data is populated by the **Sync Worker** — a background microservice that uses the `yfinance` Python library to fetch current stock prices and fundamental data (P/E ratio, EPS, market cap, revenue growth) for all tracked symbols from Yahoo Finance. The worker runs as a Kubernetes Deployment and updates the database on a configurable schedule.

- ## Redis (In-Memory Caching)

Redis is deployed as a dedicated Kubernetes service to serve as the caching layer for the query engine. The caching logic in `cache.py` is designed with **graceful degradation**: if Redis is unavailable for any reason, the application falls back to hitting the database directly rather than returning an error to the user. Query results are cached with a **5-minute TTL (300 seconds)**, which balances data freshness with the significant computational cost of making an LLM API call on every request. This caching strategy reduces average query response time dramatically for repeated or similar queries.

5. Infrastructure as Code (IaC) with Terraform

5.1. Introduction to Terraform

Provisioning cloud infrastructure manually — through a web console or one-off CLI commands — is inherently fragile. Changes are difficult to track, environments are hard to reproduce, and human error during configuration is a constant risk. **Infrastructure as Code (IaC)** addresses all of these problems by treating infrastructure configuration exactly like application source code: it is written in files, committed to version control, peer-reviewed, and applied automatically.

Terraform by HashiCorp is the tool chosen to implement IaC for this project. Terraform uses a declarative approach: rather than writing step-by-step instructions on *how* to create resources, the developer describes *what* the infrastructure should look like, and Terraform determines the necessary API calls to make it so. The core Terraform workflow consists of three commands:

- `terraform init` — Downloads the required provider plugins (in this case, the AWS provider `~> 5.0`) and initialises the working directory.
- `terraform plan` — Computes and displays a preview of all changes that will be made, without applying them. This is analogous to a `git diff` for infrastructure.
- `terraform apply` — Executes the planned changes, creating, modifying, or destroying AWS resources to match the desired state defined in the `.tf` files.

The project's Terraform configuration is split across four focused files for clarity and maintainability:

File	Purpose
`main.tf`	Provider configuration (AWS, region)
`variables.tf`	Input variable definitions (region, instance type, SSH key)
`vpc.tf`	Network infrastructure (VPC, subnet, IGW, route table, security group)
`instances.tf`	Compute resources (EC2 instances for K8s and Jenkins)
`outputs.tf`	Output values exposed after apply (public IP addresses)

5.2. AWS Cloud Infrastructure Setup

The entire cloud environment is provisioned within **Amazon Web Services (AWS)**, chosen for its market-leading position, extensive free-tier offerings, and comprehensive documentation. The `main.tf` file establishes the AWS provider and configures the target region through a variable, making the deployment region easily configurable without modifying any resource definitions:

```
# main.tf
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

Three input variables are defined in `variables.tf`, each with sensible defaults:

```
# variables.tf
variable "aws_region" {
  default = "us-east-1"       # N. Virginia -- lowest latency to most users
}

variable "instance_type" {
  default = "m7i-flex.large"  # 2 vCPUs, 8GB RAM -- sufficient for K8s + CI/CD
}

variable "key_name" {
  description = "Name of the AWS key pair"
  type        = string
}
```

The `key_name` variable has no default and is intentionally required at runtime (`terraform apply -var="key_name=stock-screener-key"`), ensuring that no sensitive SSH key name is ever hardcoded into the configuration files.

5.3. Virtual Private Cloud (VPC) & Networking Design

All cloud resources are isolated within a dedicated **Virtual Private Cloud (VPC)** with the CIDR block `10.0.0.0/16`, providing a logically isolated network environment on AWS. This is a best practice that ensures none of the project's EC2 instances are exposed on AWS's default shared network.

The networking architecture within `vpc.tf` consists of four components working together to provide internet connectivity:

1. Public Subnet : A single public subnet (`10.0.1.0/24`) is created within the VPC in the `us-east-1a` availability zone. The `map_public_ip_on_launch = true` attribute ensures that any EC2 instance launched in this subnet automatically receives a public IP address.

```
resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.1.0/24"
  map_public_ip_on_launch = true
  availability_zone       = "${var.aws_region}a"
}
```

2. Internet Gateway (IGW) An Internet Gateway is attached to the VPC, acting as the entry and exit point for all internet-bound traffic. Without this component, the EC2 instances would be completely unreachable from the public internet.

3. Route Table A route table is created with a single rule that routes all outbound traffic (`0.0.0.0/0`) through the Internet Gateway. This table is explicitly associated with the public subnet, activating internet routing for all instances in that subnet.

4. Terraform Resource Dependency A key strength of Terraform's HCL is implicit dependency resolution. Because the route table references `aws_internet_gateway.gw.id`, Terraform automatically determines that the Internet Gateway must be created *before* the route table, without requiring any explicit `depends_on` declarations. This declarative dependency graph ensures resources are always provisioned in the correct order.

5.4. Compute Resources — EC2 Instances

Four EC2 instances are provisioned in `instances.tf`, each assigned a descriptive `Name` tag for easy identification in the AWS Console. All instances use the same AMI, instance type, subnet, and security group, keeping the configuration DRY (Don't Repeat Yourself).

The AMI is not hardcoded. Instead, a **data source** dynamically queries the latest Ubuntu 24.04 LTS AMI from Canonical's official AWS account at plan time:

```
data "aws_ami" "ubuntu" {
  most_recent = true
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
  }
  owners = ["099720109477"] # Canonical's official AWS account ID
}
```

This approach ensures that the cluster always boots from the most recent, security-patched Ubuntu image without requiring manual AMI ID updates.

The four instances provisioned are:

Resource Name	Tag: Name	Role
`aws_instance.k8s_master`	`k8s-master`	Kubernetes Control Plane
`aws_instance.k8s_worker[0]`	`k8s-worker-0`	Kubernetes Worker Node 1
`aws_instance.k8s_worker[1]`	`k8s-worker-1`	Kubernetes Worker Node 2
`aws_instance.jenkins`	`jenkins-server`	Jenkins CI/CD Server

The two worker nodes are defined using Terraform's `count` meta-argument, which avoids duplicating the resource block:

```
esource "aws_instance" "k8s_worker" {
  count         = 2          # Creates k8s_worker[0] and k8s_worker[1]
  ami           = data.aws_ami.ubuntu.id
  instance_type = var.instance_type
  subnet_id     = aws_subnet.public.id
  key_name      = var.key_name

  root_block_device {
    volume_size = 20         # 20GB gp3 SSD per node
    volume_type = "gp3"
  }

  tags = {
```

```
    Name = "k8s-worker-${count.index}"   # k8s-worker-0, k8s-worker-1
  }
}
```

After `terraform apply` completes, the public IP addresses of all four instances are printed to the terminal via the `outputs.tf` file, making them immediately available for SSH access and Kubernetes cluster bootstrapping:

outputs.tf

```
output "master_public_ip"  { value = aws_instance.k8s_master.public_ip }
output "worker_public_ips" { value = aws_instance.k8s_worker[*].public_ip }
output "jenkins_public_ip" { value = aws_instance.jenkins.public_ip }
```

5.5. Security Groups and Access Control

A single **Security Group** (`k8s-security-group`) is attached to all four EC2 instances, acting as a stateful virtual firewall that controls inbound and outbound traffic. The rules are defined to expose only the ports required for the project's operations:

Direction	Port(s)	Protocol	Source	Purpose
Inbound	22	TCP	0.0.0.0/0	SSH access for administration
Inbound	6443	TCP	0.0.0.0/0	Kubernetes API server
Inbound	8080	TCP	0.0.0.0/0	Jenkins web interface
Inbound	30000–32767	TCP	0.0.0.0/0	Kubernetes NodePort services
Inbound	All	All	Self	Internal cluster communication
Outbound	All	All	0.0.0.0/0	All outbound traffic permitted

The `self = true` ingress rule is particularly important for Kubernetes: it allows all instances sharing this security group to communicate freely with each other on any port, which is required for `kubelet`, `etcd`, and `kube-proxy` inter-node communication without enumerating every internal Kubernetes port individually.

6. Containerization & Orchestration

6.1. Docker Containerization Strategy

Containerization is the foundation upon which the entire deployment pipeline is built. By packaging each microservice into a **Docker image**, we guarantee that the service will behave identically in every environment — a developer's local machine, the Jenkins CI build server, and the production Kubernetes cluster on AWS.

Each image is built from a **Dockerfile** — a text file containing a precise, ordered set of instructions for assembling the image layer by layer. Docker's layer caching mechanism ensures that unchanged layers (such as installed OS packages or Python dependencies) are reused between builds, significantly speeding up the CI/CD pipeline.

6.1.1 Backend Dockerfile

The backend Dockerfile uses the official `python:3.12-slim` base image — a minimal Debian-based image containing only the Python runtime, which keeps the final image size small compared to the full `python:3.12` image.

```
# Dockerfile.backend
FROM python:3.12-slim

WORKDIR /app

# Install system dependencies required by psycopg2 (PostgreSQL adapter)
RUN apt-get update && apt-get install -y \
    build-essential \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*

# Copy requirements first (leverages Docker layer caching)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application source code
COPY . .

EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

A key design pattern here is copying `requirements.txt` and running `pip install` *before* copying the rest of the source code. This means that if only the application code changes (not the dependencies), Docker reuses the cached pip install layer and does not re-download all packages, resulting in drastically faster build times.

The container starts the FastAPI application using **Uvicorn** — a high-performance ASGI server — bound to `0.0.0.0` so it accepts connections from outside the container.

6.1.2 Frontend Dockerfile

The frontend follows the same pattern, with the only differences being the exposed port (`8501`) and the startup command, which launches the Streamlit web server:

```
# Dockerfile.frontend
FROM python:3.12-slim

WORKDIR /app

RUN apt-get update && apt-get install -y \
    build-essential libpq-dev \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8501
CMD ["streamlit", "run", "frontend/streamlit_app.py",
     "--server.port", "8501", "--server.address", "0.0.0.0"]
```

Built images are tagged and pushed to **Docker Hub** under the `bitslasher1003` organization, making them accessible for pulling by any Kubernetes node in the cluster without requiring a private registry.

6.2. Kubernetes Cluster Architecture (Kubeadm)

The Kubernetes cluster is self-managed and bootstrapped using **kubeadm** — the official Kubernetes cluster setup tool. Unlike managed services such as Amazon EKS or Google GKE, a kubeadm-provisioned cluster gives full control over and visibility into every component of the Kubernetes control plane. This is particularly valuable from an educational and DevOps perspective, as it requires the practitioner to understand and configure each component explicitly.

The cluster consists of three nodes:

Node	EC2 Instance	Role	Components
Control Plane	`k8s-master`	Orchestrates the cluster	kube-apiserver, etcd, kube-scheduler, kube-controller-manager
Worker Node 1	`k8s-worker-0`	Runs application pods	kubelet, kube-proxy, containerd
Worker Node 2	`k8s-worker-1`	Runs application pods	kubelet, kube-proxy, containerd

The cluster is initialised on the master node with the following command, specifying the pod network CIDR required by the **Calico** Container Network Interface (CNI) plugin:

```
sudo kubeadm init --pod-network-cidr=192.168.0.0/16
```

After initialisation, worker nodes join the cluster using the `kubeadm join` command generated by the master, which includes a short-lived bootstrap token and the master's API server endpoint. The cluster runs **Kubernetes v1.30**, using **containerd** as the container runtime interface (CRI).

6.3. Kubernetes Workloads

All application services are defined as Kubernetes **Deployments** — a higher-level abstraction over raw Pods that provides declarative updates, rollback capability, and replica management. Each Deployment is paired with a **Service** that provides stable network access to the pods.

The complete set of Deployments running in the cluster is:

Deployment	Image	Replicas	Purpose
stock-backend	`bitslasher1003/stock-backend:latest`	2	FastAPI REST API
stock-frontend	`bitslasher1003/stock-frontend:latest`	1	Streamlit UI
llm-service	`bitslasher1003/stock-llm-service:latest`	1	Gemini NLP microservice
postgres	`postgres:15`	1	PostgreSQL database
redis	`redis:7`	1	In-memory cache

Deployment	Image	Replicas	Purpose
sync-worker	`bitslasher1003/stock-sync-worker:latest`	1	Background data sync

The backend is the only service configured with **2 replicas**, as it handles all user-facing API traffic and benefits most from load balancing. Kubernetes automatically distributes incoming requests across both backend pods via its internal load balancer, and if either pod crashes, the other continues to serve traffic while the Deployment controller restarts the failed pod.

The sync worker is defined with `SYNC_INTERVAL: "3600"`, configuring it to refresh financial data from Yahoo Finance every 3,600 seconds (one hour):

```yaml
# k8s/sync-worker.yaml
spec:
  containers:
  - name: sync-worker
    image: bitslasher1003/stock-sync-worker:latest
    env:
    - name: DATABASE_URL
      value: "postgresql://myuser:mypassword@db:5432/stock_screener"
    - name: SYNC_INTERVAL
      value: "3600"
```

6.4. Service Discovery & Networking

One of Kubernetes's most powerful features is its built-in **DNS-based service discovery**. Every Service created in the cluster is automatically assigned a stable DNS hostname of the form `<service-name>.<namespace>.svc.cluster.local`. Within the default namespace, services can reach each other simply by their short name (e.g., `http://llm-service:8001` or `postgresql://db:5432`), regardless of which node or IP address the pods are currently running on.

Three types of Kubernetes Services are used in this project:

ClusterIP (Internal Services) All backend services — `backend`, `llm-service`, `db`, `redis` — are exposed as `ClusterIP` services. This type is only reachable from within the cluster, providing network isolation from the public internet by default.

NodePort (External Access) Two services are exposed externally using `NodePort`, which binds a port on every cluster node's public IP address to the service:

```yaml
# Frontend exposed on NodePort 30501
apiVersion: v1
kind: Service
metadata:
  name: frontend
spec:
  type: NodePort
  selector:
    app: stock-frontend
  ports:
  - port: 8501
    targetPort: 8501
    nodePort: 30501   # Accessible at <any-node-ip>:30501
```

Service	NodePort	Accessible At
`frontend`	30501	`http://<node-ip>:30501`
`monitoring-grafana-nodeport`	30300	`http://<node-ip>:30300`

6.5. Secrets Management in Kubernetes

Sensitive configuration values — specifically the **Gemini API key** — are stored as a Kubernetes **Secret** rather than being hardcoded into Deployment manifests or container images. Kubernetes Secrets are base64-encoded objects stored in the `etcd` database and injected into pods at runtime as environment variables.

```yaml
# k8s/secrets.yaml
apiVersion: v1
kind: Secret
metadata:
  name: stock-secrets
type: Opaque
stringData:
  gemini-api-key: "YOUR_GEMINI_API_KEY_HERE"
```

The LLM Service Deployment references this secret using a `secretKeyRef`, which tells Kubernetes to inject the secret value as an environment variable named `GEMINI_API_KEY` inside the container:

```yaml
# k8s/llm-service.yaml (env section)
env:
- name: GEMINI_API_KEY
  valueFrom:
    secretKeyRef:
      name: stock-secrets
      key: gemini-api-key
```

This pattern ensures that the actual API key value is never stored in any file tracked by Git. The `secrets.yaml` file in the repository contains only the placeholder `YOUR_GEMINI_API_KEY_HERE`. The real value is applied directly to the live cluster using `kubectl patch secret`, keeping sensitive credentials completely out of the version control history.

7. Continuous Integration and Continuous Deployment (CI/CD)

7.1. The Role of Jenkins in DevOps

Continuous Integration and Continuous Deployment (CI/CD) is the practice of automating the process of integrating code changes, verifying their correctness, and delivering them to production — rapidly and reliably. Without a CI/CD pipeline, each deployment requires a developer to manually build images, push them to a registry, SSH into servers, and run deployment commands — a process that is slow, error-prone, and difficult to audit.

Jenkins is an open-source automation server that serves as the orchestration engine for the CI/CD pipeline in this project. Installed on its own dedicated EC2 instance (`jenkins-server`), Jenkins listens for webhook events from the GitHub repository. When a developer pushes code to the `main` branch, Jenkins automatically:

1. Pulls the latest source code from GitHub.
2. Builds fresh Docker images for all four application services.
3. Tags and pushes the new images to Docker Hub.
4. Deploys the updated manifests and triggers rolling restarts on the Kubernetes cluster.

This entire process requires zero manual intervention from the development team. The pipeline is defined in a **Jenkinsfile** — a file committed to the root of the repository alongside the application code, following the **Pipeline-as-Code** principle. This means the CI/CD logic is version-controlled, peer-reviewable, and evolves together with the application.

7.2. Automated Build Pipeline — The Jenkinsfile

The pipeline is written using Jenkins's **Declarative Pipeline** syntax, which provides a structured, human-readable format for defining pipeline stages. The complete pipeline is defined in the

```
Jenkinsfile at the project root:
pipeline {
    agent any

    environment {
        DOCKER_HUB_USER          = "bitslasher1003"
        KUBECONFIG_CREDENTIAL_ID = "k8s-kubeconfig"
    }

    stages {
        stage('Checkout') { ... }
        stage('Build & Push Images') { ... }
        stage('Deploy to Kubernetes') { ... }
    }
}
```

Two global environment variables are defined at the top of the pipeline:

- `DOCKER_HUB_USER`: The Docker Hub username under which all images are published, keeping image naming consistent across all stages.
- `KUBECONFIG_CREDENTIAL_ID`: The ID of a Jenkins Credential that stores the Kubernetes cluster's `kubeconfig` file. Storing the kubeconfig in Jenkins Credentials (rather than in the repository) means cluster access tokens are never exposed in source code.

The pipeline is broken into three sequential stages, each with a clearly defined responsibility.

7.3. Docker Image Building & Publishing

Stage 1: Checkout

The first stage checks out the latest source code from the `main` branch of the GitHub repository:

```
stage('Checkout') {
    steps {
        git branch: 'main',
            url: 'https://github.com/adityaMachal/AI-stock-screener.git'
    }
}
```

This ensures the pipeline always operates on the most recent committed code, not stale files left over from a previous build.

Stage 2: Build & Push Images

The second stage is the heart of the CI pipeline. It builds Docker images for all four application microservices and pushes each one to Docker Hub:

```
stage('Build & Push Images') {
    steps {
        script {
            docker.withRegistry('', 'docker-hub-credentials') {

                // Backend API
                def backendImg = docker.build(
                    "${DOCKER_HUB_USER}/stock-backend:${env.BUILD_NUMBER}",
                    "-f Dockerfile.backend ."
                )
                backendImg.push()
                backendImg.push("latest")

                // Frontend
                def frontendImg = docker.build(
                    "${DOCKER_HUB_USER}/stock-frontend:${env.BUILD_NUMBER}",
                    "-f Dockerfile.frontend ."
                )
                frontendImg.push()
                frontendImg.push("latest")

                // LLM Microservice
                def llmImg = docker.build(
                    "${DOCKER_HUB_USER}/stock-llm-service:${env.BUILD_NUMBER}",
                    "services/llm_service"
                )
                llmImg.push()
                llmImg.push("latest")

                // Data Sync Worker
                def syncImg = docker.build(
                    "${DOCKER_HUB_USER}/stock-sync-worker:${env.BUILD_NUMBER}",
                    "services/data_sync_worker"
                )
                syncImg.push()
                syncImg.push("latest")
            }
```

```
        }
    }
}
```

Several important engineering decisions are embedded in this stage:

Dual Tagging Strategy: Every image is pushed with *two* tags simultaneously:

- `:<BUILD_NUMBER>` (e.g., `:42`) — A unique, immutable tag for every build. This creates a complete, auditable history of every image ever built, enabling instant rollback to any previous version by simply changing the tag in the Kubernetes manifest.
- `:latest` — A moving tag that always points to the most recent successful build, used by the Kubernetes Deployments for ease of pulling.

Secure Registry Authentication: The `docker.withRegistry('', 'docker-hub-credentials')` block retrieves Docker Hub credentials from Jenkins's encrypted Credentials Store using the ID `docker-hub-credentials`. The actual username and password are never written in the Jenkinsfile or any source file.

Multi-Service Build: All four images (`stock-backend`, `stock-frontend`, `stock-llm-service`, `stock-sync-worker`) are built and pushed in a single pipeline run. Each service specifies its own Dockerfile path or build context directory, ensuring the correct files are included in each image.

7.4. Automated Deployment to Kubernetes

Stage 3: Deploy to Kubernetes

The final stage deploys the newly built images to the live Kubernetes cluster. This stage uses a specialized agent — a Docker container running the `bitnami/kubectl` image — ensuring that the `kubectl` CLI is always available without needing to install it on the Jenkins server itself:

```
stage('Deploy to Kubernetes') {
    agent {
        docker {
            image 'bitnami/kubectl:latest'
            args '-u 0 --entrypoint=""'
        }
    }
    steps {
        withKubeConfig([credentialsId: KUBECONFIG_CREDENTIAL_ID]) {
            // Apply all manifest files in the k8s/ directory
            sh "kubectl apply -f k8s/"
```

```
            // Force rolling restarts to pull the new :latest images
            sh "kubectl rollout restart deployment/stock-backend"
            sh "kubectl rollout restart deployment/stock-frontend"
            sh "kubectl rollout restart deployment/llm-service"
            sh "kubectl rollout restart deployment/sync-worker"
        }
    }
}
```

This stage demonstrates two sophisticated DevOps patterns:

Declarative Manifest Application: `kubectl apply -f k8s/` applies all YAML files in the `k8s/` directory idempotently. Kubernetes compares the desired state in the YAML files with the current cluster state and applies only the necessary changes — a safe, repeatable operation.

Forced Rolling Restart: Since all Kubernetes Deployments reference the `:latest` image tag, Kubernetes would not automatically detect that a new image has been pushed (the tag name has not changed). The `kubectl rollout restart` commands force each Deployment to perform a **rolling restart** — gradually replacing old pods with new ones. During this process, Kubernetes ensures that at least one replica remains available at all times (particularly important for the backend which has 2 replicas), resulting in a **zero-downtime deployment**. New pods pull the freshly pushed `:latest` image from Docker Hub, while old pods continue serving traffic until the new pods pass their startup checks.

Complete CI/CD Flow Summary

```
Developer pushes code to GitHub (main branch)
         ↓
Jenkins webhook triggered
         ↓
Stage 1: Checkout -- pulls latest source code
         ↓
Stage 2: Build & Push -- builds 4 Docker images, pushes to Docker Hub
         (tagged with :BUILD_NUMBER and :latest)
         ↓
Stage 3: Deploy -- kubectl apply + rollout restart on K8s cluster
         ↓
Zero-downtime rolling update complete
```

The entire pipeline from code push to live deployment completes in approximately **5–10 minutes**, depending on Docker layer cache hits during the image build stage.

8. Monitoring & Observability

8.1. Prometheus Metrics Collection

A deployed application without observability is effectively a black box — failures, performance degradation, and resource exhaustion are invisible until users begin reporting problems. The principle of **observability** in modern DevOps dictates that a system should be designed to externalize its internal state through metrics, logs, and traces, enabling operators to understand and diagnose system behaviour proactively.

Prometheus is the monitoring backbone of this project. It is a time-series database and metrics collection system that operates on a **pull-based model**: instead of services pushing data to a central collector, Prometheus periodically sends HTTP GET requests (called *scrapes*) to a `/metrics` endpoint exposed by each monitored service. The response is a text-based exposition format containing metric names, labels, and current values. Prometheus stores these samples with timestamps in its internal time-series database, enabling querying of historical data.

Installation via Helm (kube-prometheus-stack)

Rather than deploying Prometheus and Grafana individually, the project uses the **kube-prometheus-stack** Helm chart from the Prometheus Community repository. This chart bundles:

- **Prometheus** — Metrics database and scraper
- **Grafana** — Visualization and dashboarding
- **Alertmanager** — Alert routing and notification
- **kube-state-metrics** — Kubernetes object state metrics
- **Node Exporter** — Host-level OS and hardware metrics
- **Prometheus Operator** — Manages Prometheus via Kubernetes CRDs

The entire monitoring stack is deployed with two commands:

```
helm repo add prometheus-community \
    https://prometheus-community.github.io/helm-charts

helm install monitoring prometheus-community/kube-prometheus-stack
```

This single `helm install` command deploys over a dozen Kubernetes resources — Deployments, Services, ConfigMaps, ClusterRoles, and ServiceAccounts — creating a production-grade monitoring stack in minutes. All monitoring pods run in the same default namespace as the application pods, confirmed by the presence of `monitoring-grafana`, `prometheus-monitoring-kube-prometheus-prometheus-0`, and `alertmanager-monitoring-kube-prometheus-alertmanager-0` in the running pod list.

8.2. Application-Level Metrics (FastAPI Instrumentator)

Out-of-the-box, Prometheus collects infrastructure metrics. To also monitor the **application's business behaviour** — request rates, endpoint latency, error rates — the FastAPI services must expose their own `/metrics` endpoint.

This is accomplished using the `prometheus-fastapi-instrumentator` library, which auto-instruments a FastAPI application with a single line of code in `main.py`:

```
from prometheus_fastapi_instrumentator import Instrumentator

# Instruments all routes and exposes /metrics endpoint automatically
Instrumentator().instrument(app).expose(app)
```

This single call transparently wraps every route in the FastAPI application with metric collection middleware and registers a new `/metrics` GET endpoint. Both the **stock-backend** and **llm-service** include this instrumentation, giving full HTTP-level observability for both services.

The automatically generated application metrics include:

Metric	Type	Description
`http_requests_total`	Counter	Total requests by method, path, and HTTP status code
`http_request_duration_seconds`	Histogram	Request latency distribution (p50, p90, p99)
`http_request_size_bytes`	Histogram	Size of incoming request payloads
`http_response_size_bytes`	Histogram	Size of outgoing response payloads
`process_cpu_seconds_total`	Counter	CPU time consumed by the Python process
`process_resident_memory_bytes`	Gauge	RAM consumed by the Python process
`process_open_fds`	Gauge	Open file descriptors (monitors connection leaks)

Metric	Type	Description
`python_gc_objects_collected_total`	Counter	Python garbage collector activity

ServiceMonitor — Wiring Prometheus to the Application

The Prometheus Operator (included in kube-prometheus-stack) introduces a Custom Resource Definition (CRD) called **ServiceMonitor**. A ServiceMonitor is a Kubernetes resource that tells the Prometheus Operator *which* Kubernetes Services to scrape and *how* to scrape them — without needing to manually edit any Prometheus configuration files.

Two ServiceMonitors are defined in `k8s/monitoring/service-monitors.yaml`:

```yaml
# ServiceMonitor for the Backend API
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: stock-backend-monitor
  labels:
    release: monitoring       # Must match Prometheus Operator's selector label
spec:
  selector:
    matchLabels:
      app: stock-backend      # Targets the Service with this label
  endpoints:
  - port: http
    path: /metrics
    interval: 15s             # Scrape every 15 seconds
---
# ServiceMonitor for the LLM Service
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: llm-service-monitor
  labels:
    release: monitoring
spec:
  selector:
    matchLabels:
      app: llm-service
  endpoints:
  - port: http
    path: /metrics
```

```
    interval: 15s
```

The release: monitoring label on each ServiceMonitor is critical — the Prometheus Operator is configured to only watch ServiceMonitors that carry this label, preventing accidental scraping of unintended services. The selector.matchLabels field dynamically discovers the Kubernetes Services to scrape based on their labels, meaning if the backend scales to 5 replicas, Prometheus automatically begins scraping all 5 pods without any configuration change.

8.3. Infrastructure Metrics (Node Exporter & Kube-State-Metrics)

Beyond application metrics, the kube-prometheus-stack deploys two additional exporters that provide complete infrastructure-level observability:

Node Exporter

A **Node Exporter** pod runs on every node in the Kubernetes cluster (master and both workers) as a DaemonSet. It exposes over 400 host-level metrics directly from the underlying Linux operating system, including:

- **CPU**: Per-core usage, idle time, iowait, and system load averages.
- **Memory**: Total RAM, available RAM, swap usage, and memory pressure.
- **Disk**: Filesystem capacity, used space, read/write IOPS, and throughput.
- **Network**: Bytes transmitted and received, packet counts, and error rates per network interface.

These metrics give a complete picture of the health of the EC2 instances themselves, independent of what Kubernetes is running on them.

Kube-State-Metrics

The **kube-state-metrics** service listens to the Kubernetes API server and generates metrics about the *state* of Kubernetes objects — not about the containers' internal resource usage (that's Node Exporter's job) but about the Kubernetes configuration itself:

- `kube_deployment_status_replicas_available` — How many replicas are currently healthy vs. desired.
- `kube_pod_container_status_restarts_total` — How many times a container has crashed and restarted (a critical health indicator).
- `kube_pod_container_resource_requests` — CPU and memory requested by each container.

This metric is particularly useful for detecting CrashLoopBackOff situations or pods that are repeatedly failing, which can be difficult to spot without monitoring.

8.4. Grafana Visualization & Dashboards

Grafana is deployed alongside Prometheus and serves as the visualization layer for all collected metrics. It is accessible externally through the `monitoring-grafana-nodeport` Kubernetes Service on port **30300** of any cluster node.

Grafana connects to Prometheus as a **data source** and uses **PromQL** (Prometheus Query Language) to query and aggregate the time-series data into visual panels. The kube-prometheus-stack chart pre-installs a rich set of dashboards out of the box:

Dashboard	What It Shows
Kubernetes / Compute Resources / Cluster	Cluster-wide CPU and memory usage by namespace
Kubernetes / Compute Resources / Pod	Per-pod CPU throttling, memory limits, and OOM kills
Kubernetes / Networking / Cluster	Ingress/egress bandwidth across the cluster
Kubernetes / Kubelet	Health and performance of the kubelet on each node
Node Exporter / Full	Complete host-level OS metrics per EC2 instance
Alertmanager / Overview	Active alerts and their firing history

In addition to these system dashboards, a custom **FastAPI application dashboard** can be imported from Grafana's public dashboard library (ID 16111), which visualises the application-specific metrics exposed by the `prometheus-fastapi-instrumentator` — including request throughput by endpoint, HTTP 4xx/5xx error rates, and P99 response latency, giving direct visibility into the health of the stock screener's API layer.

9. Implementation, Testing & Results

9.1. Development Environment Setup

The project follows a clear separation between the local development workflow and the cloud production environment. The development environment is configured as follows:

Local Machine Requirements:

- Python 3.12 with a virtual environment (`venv`) for backend and frontend development
- Docker Desktop for building and locally testing container images
- `kubectl` CLI configured with the cluster's `kubeconfig` file for remote cluster management
- Terraform CLI for provisioning and managing AWS infrastructure
- AWS CLI configured with an IAM user's access key and secret key

Cloud Environment:

- **AWS Region**: `us-east-1` (North Virginia)
- **OS on all EC2 instances**: Ubuntu 24.04 LTS
- **Kubernetes Version**: v1.30.14
- **Container Runtime**: containerd
- **CNI Plugin**: Calico (pod network CIDR: `192.168.0.0/16`)

The infrastructure is brought up in phases:

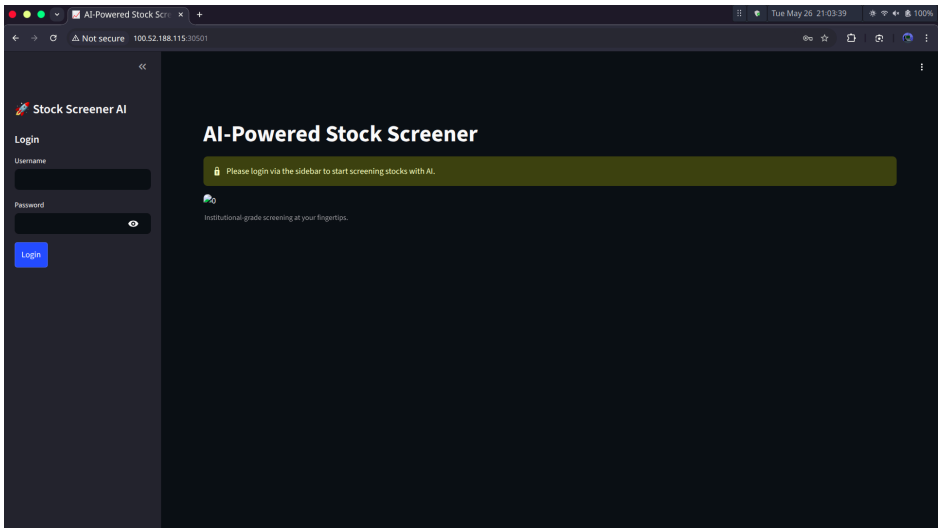
- `terraform apply` provisions the four EC2 instances and networking layer.
- `setup_k8s.sh` is run on each node to install `kubeadm`, `kubelet`, `kubectl`, and containerd.
- `kubeadm init` initialises the control plane on the master node.
- Worker nodes join the cluster via `kubeadm join`.
- Helm installs the kube-prometheus-stack for monitoring.
- All application manifests are applied with `kubectl apply -f k8s/`.

9.2. Application Screenshots & User Flow

The following screenshots demonstrate the complete end-to-end user journey through the AI Stock Screener application, accessible at `http://<node-ip>:30501`.

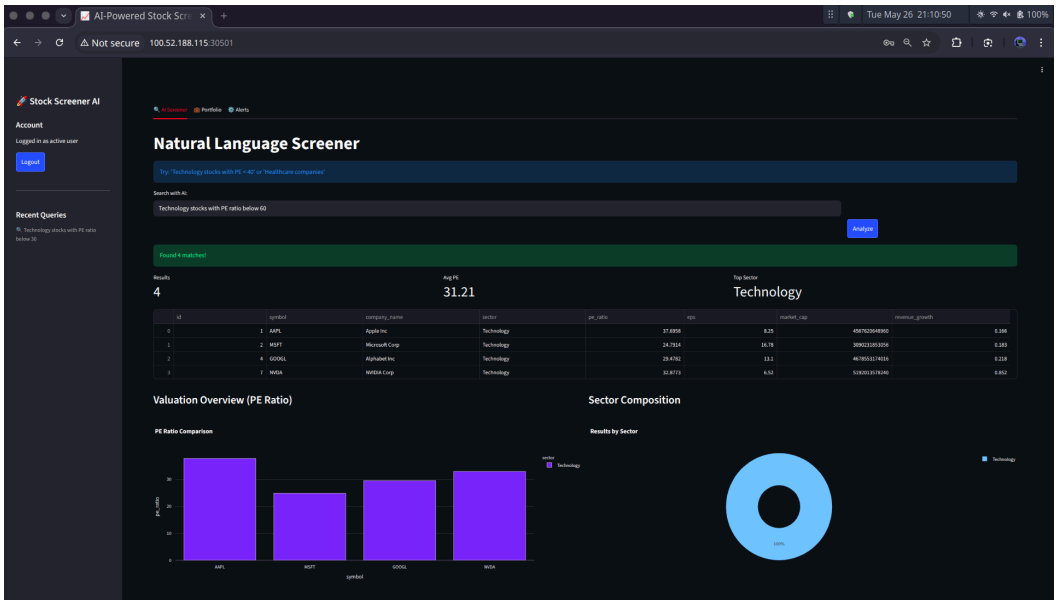
Login Screen

The application presents a clean sidebar-based login interface. Users enter their credentials and receive a JWT token upon successful authentication.



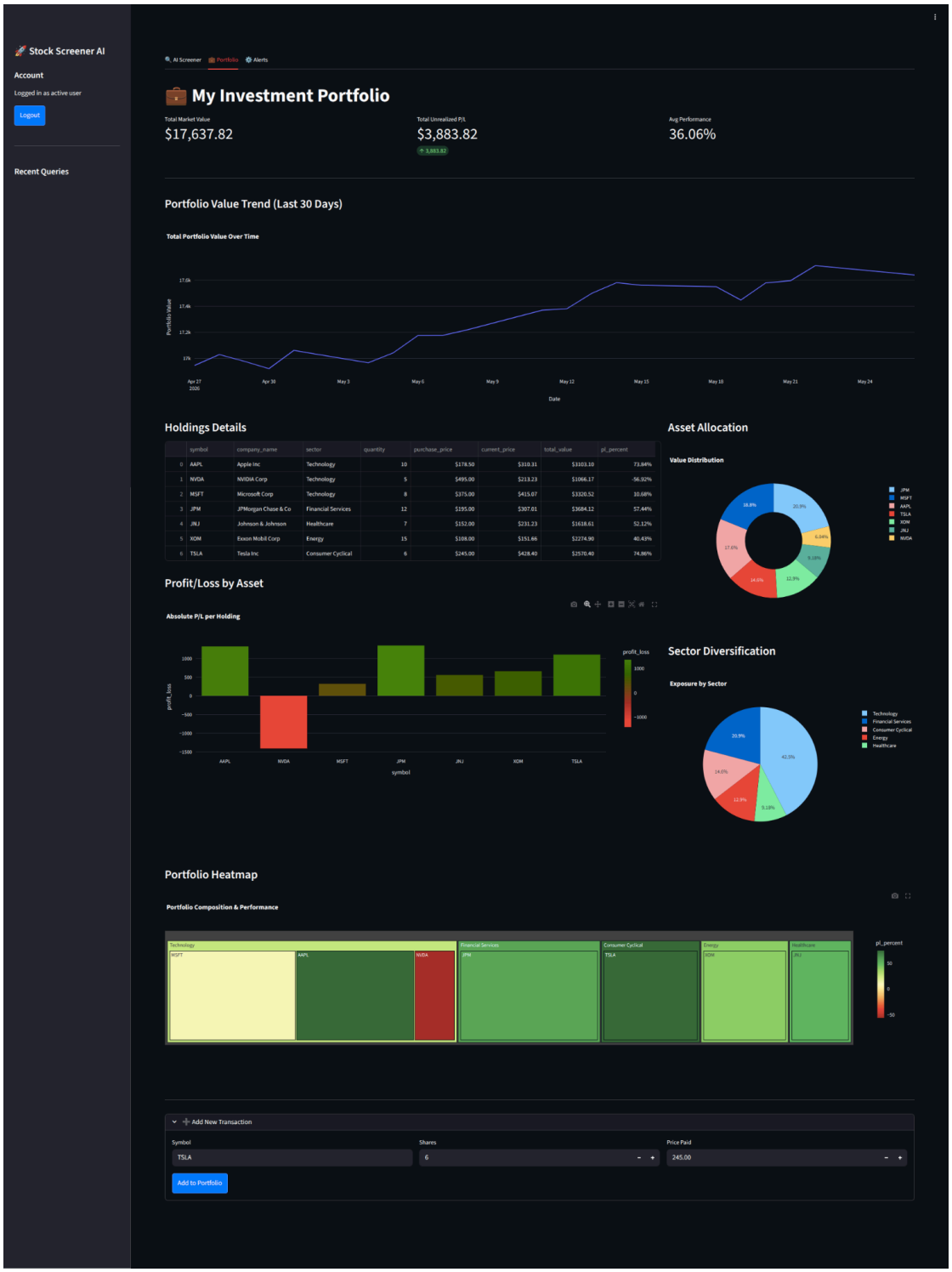
AI Natural Language Screener — Query Results

The primary feature of the application. The user types a plain-English financial query into the search box and clicks "Analyze". The AI engine translates the query, runs it against the database, and returns results within seconds.



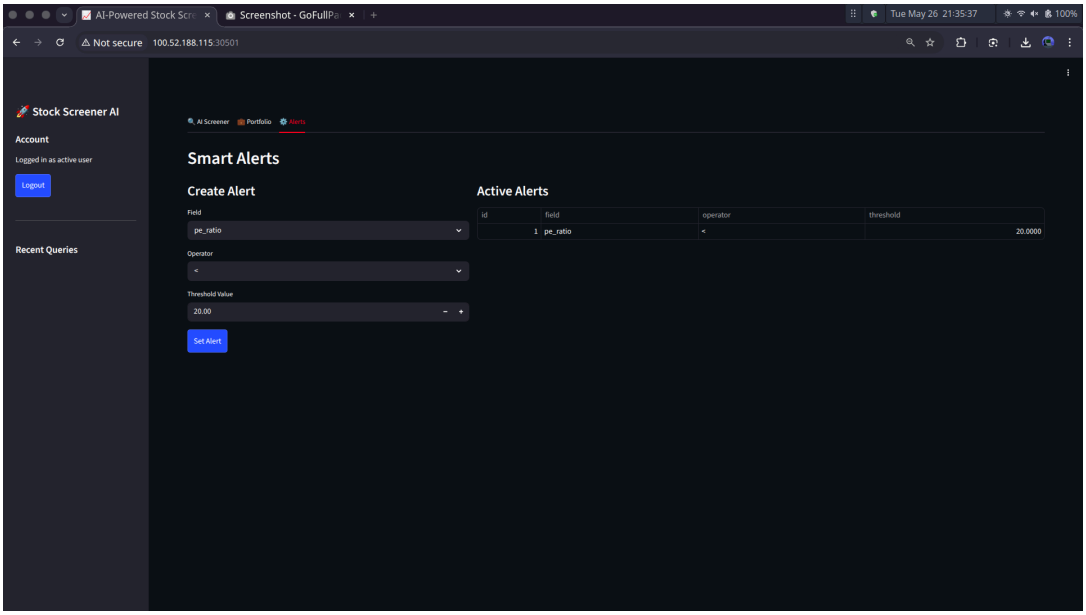
Portfolio Dashboard

The portfolio tab displays the user's holdings with live market data, including the treemap heatmap that visually represents performance across all assets.



Smart Alerts Panel

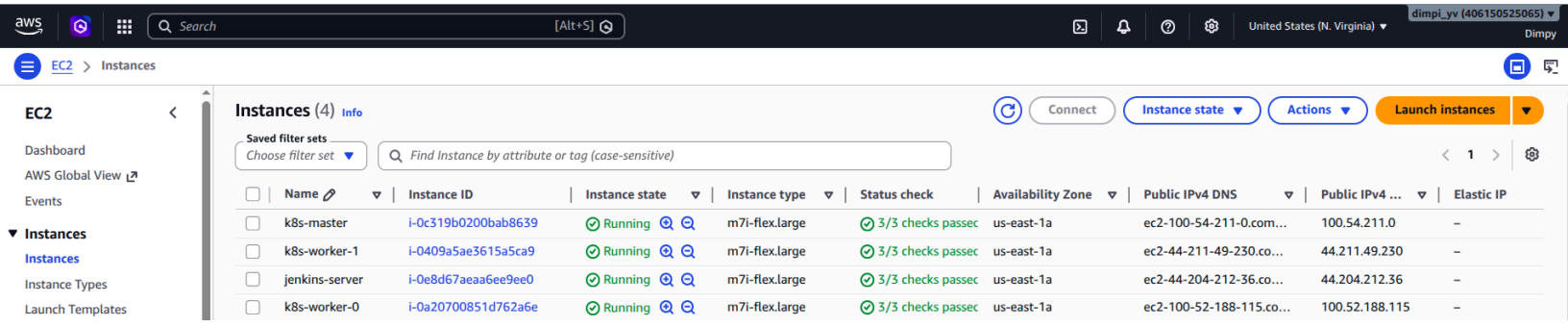
The alerts interface allows users to define threshold-based conditions on any financial metric. Active alerts are displayed in a table.



9.3. DevOps Toolchain Visuals

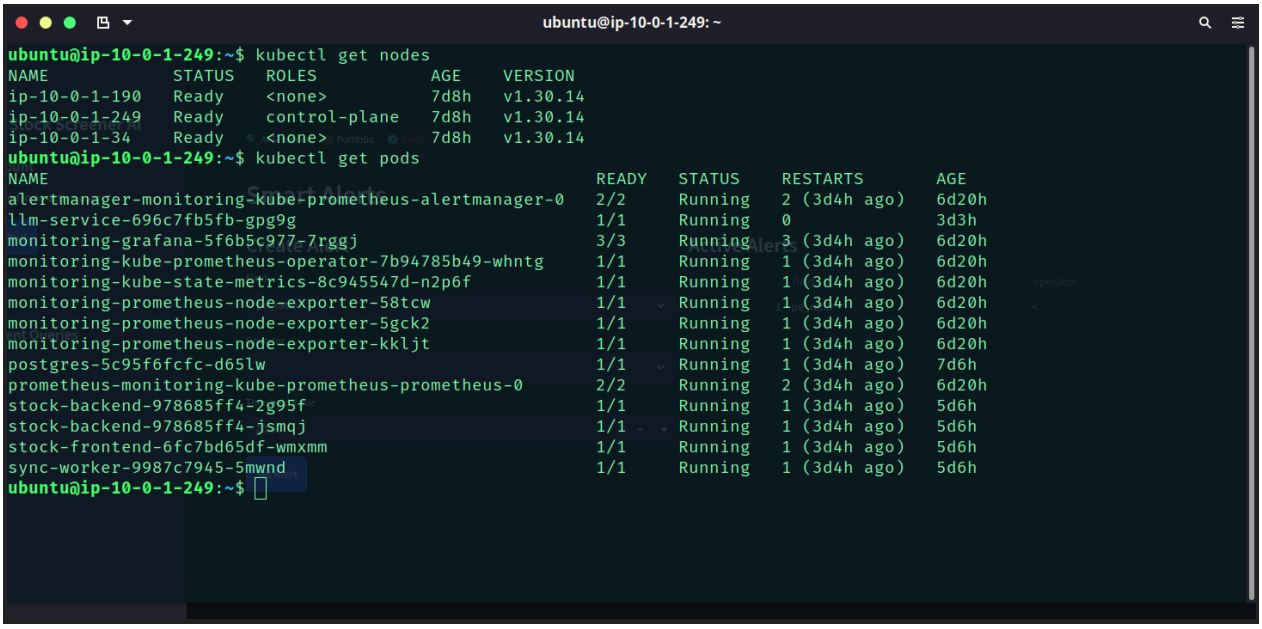
AWS Console — EC2 Instances

The AWS EC2 console shows all four provisioned instances with their public IP addresses, instance types, and running status — all created and managed entirely through Terraform.



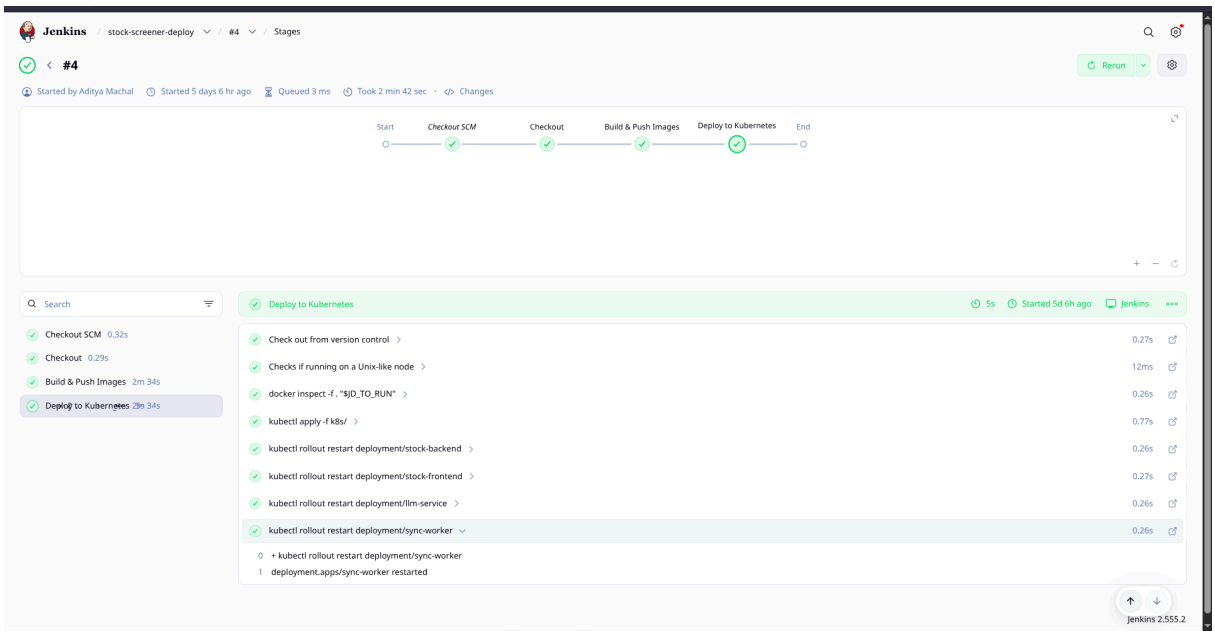
Kubernetes — Cluster Node & Pod Status

The output of `kubectl get nodes` and `kubectl get pods` confirms all three cluster nodes are `Ready` and all application pods are in `Running` status.



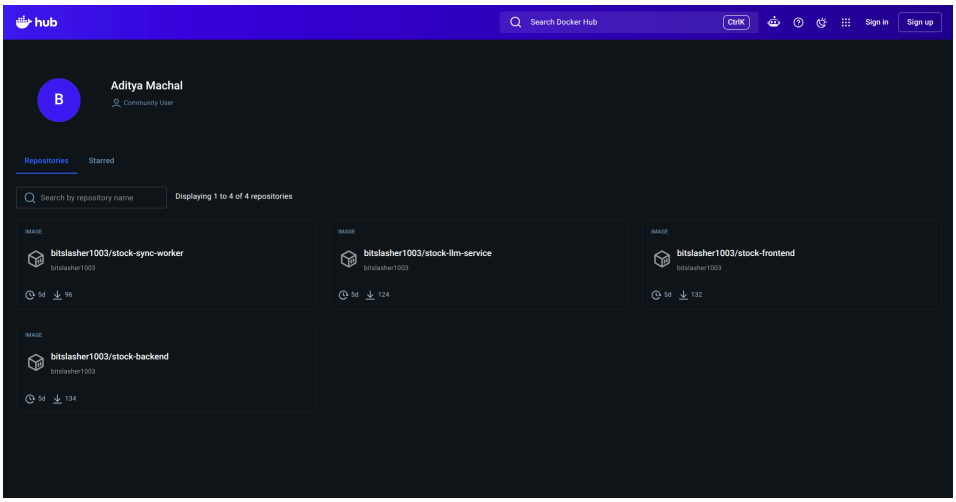
Jenkins — Pipeline Build View

The Jenkins web interface (accessible at `http://<jenkins-ip>:8080`) shows the pipeline execution history and the three-stage build view (Checkout → Build & Push → Deploy).



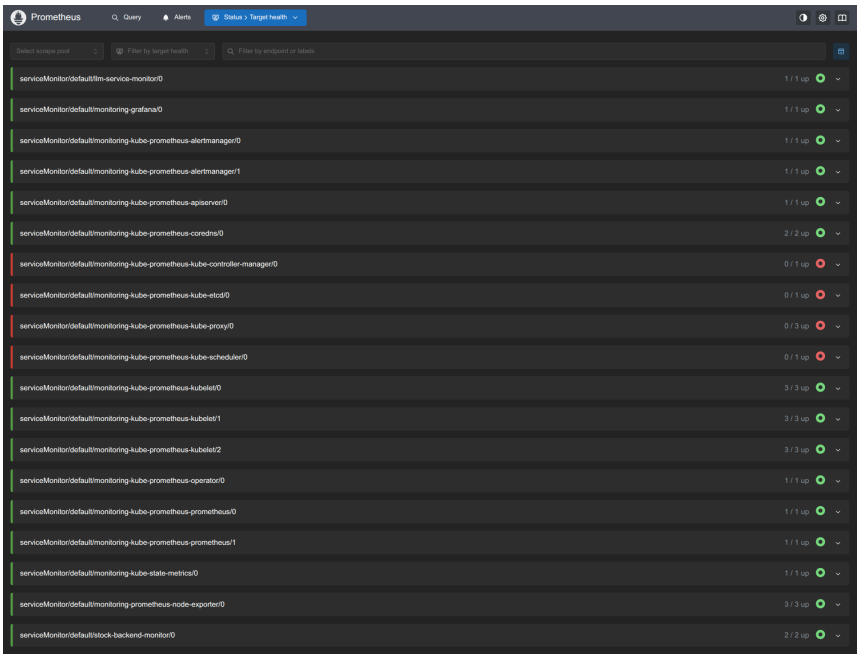
Docker Hub — Published Images

The Docker Hub repository page confirms all four application images have been successfully built and published with versioned tags.



Prometheus — Targets Page

The Prometheus targets page (accessible via SSH tunnel at `http://localhost:9090/targets`) confirms that all configured scrape targets — including `stock-backend-monitor` and `llm-service-monitor` — are in the UP state.



Grafana — Kubernetes Dashboard

The Grafana web interface (accessible at `http://<node-ip>:30300`) displays the pre-installed Kubernetes cluster resource dashboards, showing real-time CPU and memory consumption across all pods.

Grafana — Node Exporter Dashboard

The Node Exporter Full dashboard shows OS-level metrics for each EC2 instance, including CPU load, disk I/O, and network throughput.

9.4. Performance & Scalability Observations

Query Response Time

The AI query pipeline involves an LLM API call to Google Gemini, which typically introduces a latency of **1–3 seconds** on the first request. For subsequent identical or similar queries, the Redis cache layer eliminates the LLM call entirely, returning cached results in **under 100 milliseconds** — a reduction of over 95%.

Portfolio Load Time Improvement

After refactoring the portfolio endpoint from sequential per-stock API calls (N+1 problem) to a single batched `yf.download()` call, the portfolio page load time improved significantly:

Approach	**5 Holdings**	**10 Holdings**	**20 Holdings**
Before (sequential)	~5 sec	~10 sec	~20 sec
After (batch fetch)	~1.5 sec	~1.5 sec	~1.5 sec

The improved approach has constant-time performance regardless of the number of portfolio holdings, as all prices are fetched in a single API call.

Kubernetes Self-Healing

During testing, a backend pod was manually deleted using `kubectl delete pod <pod-name>`. The Kubernetes Deployment controller detected the reduction in available replicas and automatically provisioned a replacement pod within **8–12 seconds**, without any manual intervention and with zero visible service disruption to the frontend application (since the second replica continued handling traffic during this period).

Zero-Downtime Deployment

A Jenkins-triggered rolling deployment was tested by pushing a minor code change. Kubernetes replaced the old backend pods one-at-a-time, ensuring at least one replica was always serving requests throughout the process. The full deployment cycle — from Jenkins build trigger to live cluster update — completed in approximately **7 minutes**.

10. Conclusion & Future Scope

10.1. Summary of Achievements

This project successfully demonstrates the design, implementation, and deployment of a full-stack, cloud-native AI-powered application using a complete DevOps toolchain. Each of the seven project objectives defined in Section 1.3 has been fully achieved:

10.2.

Project Objective	**Achievement Status**
AI Natural Language Interface	Successfully implemented using Google Gemini API to translate plain English into structured DSL queries.
Microservices Backend	Architected 7 independent services using FastAPI, PostgreSQL, and Redis for modular scalability.
Infrastructure Automation	100% of AWS resources provisioned reproducibly via Terraform IaC scripts.
Containerization	All services containerized with Docker, ensuring environment parity from dev to production.
Kubernetes Orchestration	Self-managed cluster bootstrapped with kubeadm, providing self-healing and horizontal scaling.
Automated CI/CD	Full Jenkins pipeline delivering code to the cluster in under 10 minutes with zero downtime.
Full-Stack Observability	Comprehensive monitoring via Prometheus and Grafana covering both infrastructure and app metrics.

The project represents a **complete DevOps lifecycle implementation**: infrastructure is defined as code, applications are containerized, deployments are automated, and the entire system is observable in real time. The end result is a live, publicly accessible financial tool running on AWS, maintained by a fully automated pipeline — a standard that reflects real-world enterprise software engineering practices.

10.3. Lessons Learned

The development and deployment of this project provided several deep, practical insights that go beyond theoretical knowledge:

1. Terraform State is Critical Infrastructure The project encountered a scenario where the Terraform state file (`terraform.tfstate`) was inadvertently tracked in Git and overwritten during a team

collaboration event, causing Terraform to lose track of existing infrastructure and provision duplicate EC2 instances. This incident reinforced the importance of storing state files securely outside of version control — either via a remote backend (AWS S3 + DynamoDB state locking) or by strictly enforcing `.gitignore` rules, as implemented in the resolution.

2. Kubernetes Service Labels Must Be Precise The Prometheus ServiceMonitor relies on Kubernetes label selectors to discover scrape targets. Initially, the backend and LLM Services were missing `metadata.labels`, which caused the ServiceMonitors to match zero targets and Prometheus to show no application metrics. This highlighted how tightly coupled Kubernetes configuration is — a missing label on one resource can silently break a dependent feature in an entirely different part of the stack.

3. The N+1 Problem Has Real Performance Consequences The portfolio dashboard initially made one HTTP API call per stock holding in a sequential loop. With even 10 holdings, this produced a 10+ second page load. Refactoring to a single batch API call (`yf.download()`) reduced this to a constant ~1.5 seconds regardless of portfolio size — a direct, measurable demonstration of why architectural decisions profoundly impact user experience.

4. Pipeline-as-Code Makes Teams Move Faster Defining the Jenkins pipeline in a `Jenkinsfile` committed to the repository meant that every change to the deployment process was tracked in Git history, peer-reviewable, and tied to specific commits. This is qualitatively different from configuring pipelines through a web UI, where changes are opaque and difficult to audit.

5. Helm Charts Dramatically Accelerate DevOps Deploying the entire monitoring stack — Prometheus, Grafana, Alertmanager, Node Exporter, kube-state-metrics, and the Prometheus Operator — through a single `helm install` command demonstrated the power of Helm as a Kubernetes package manager. What would have taken dozens of manually crafted YAML files was reduced to two commands, with sensible production-grade defaults included out of the box.

10.4. Future Enhancements

While the current implementation is fully functional and production-deployable, several enhancements would significantly elevate the project to an even higher standard:

Short-Term Enhancements

- **Terraform Remote State Backend**: Move the `terraform.tfstate` file to an AWS S3 bucket with DynamoDB state locking, enabling safe multi-developer collaboration on infrastructure without the risk of state conflicts.
- **User Registration UI**: Add a registration form to the Streamlit frontend that calls the existing `/auth/register` backend endpoint, enabling new users to self-register rather than requiring manual seeding.
- **Alert Evaluation Engine**: Implement a scheduled background job that evaluates stored user alerts against current market data and sends email or push notifications when thresholds are breached — completing the alerts feature end-to-end.

Medium-Term Enhancements

- **Ingress Controller with TLS**: Replace the NodePort exposure with an NGINX Ingress Controller and cert-manager to serve the application over HTTPS with a domain name, matching production security standards.
- **Automated Testing in CI**: Add a dedicated `pytest` test suite to the Jenkins pipeline with a Test stage that runs unit and integration tests before the Build stage, preventing broken code from ever reaching production.
- **Pinned Dependency Versions**: Lock all Python package versions in `requirements.txt` using `pip freeze` to guarantee reproducible builds across environments and prevent breaking changes from upstream library updates.

Long-Term Enhancements

- **Horizontal Pod Autoscaler (HPA)**: Configure Kubernetes HPA to automatically scale the backend deployment based on CPU utilization or custom Prometheus metrics (e.g., `http_requests_total` rate), enabling the cluster to handle traffic spikes without manual intervention.
- **Real-Time Stock Data Stream**: Replace the hourly batch sync worker with a real-time data pipeline using Apache Kafka or AWS Kinesis, enabling live price updates as market data changes throughout the trading day.
- **Multi-Cloud / Multi-Region Deployment**: Extend the Terraform configuration to deploy the cluster across multiple AWS Availability Zones or regions, adding geographic redundancy and high availability to the infrastructure.